2026-6-3 封装红黑树实现mymap和myset

- 1. 源码分析
- 2. 第一层：封装红黑树
  - 2.1 复用红黑树框架
  - 2.2 改造insert支持插入
  - 2.3 在底层实现迭代器迭代器
    - 2.3.1 定义迭代器结构
    - 2.3.2 在红黑树中定义begin以及end:
    - 2.3.3 mymap以及myset下红黑树的封装
- 3. 第二层：实现myset与mymap
  - 3.1 mymap封装operate[]
  - 3.2 mymap的具体实现
  - 3.3 myset的具体实现
- 4. 第四层代码测试

技巧：连蒙带猜
学底层：先学功能，再了解核心是什么？看有没有什么前置知识要了解。
第二个参数决定红黑树里面存什么？底层不是写了两个红黑树，而是将红黑树写成了一个泛型？
第二个模板参数能决定，那要第一个模板参数干什么呢？
map.h & set.h的代码实现，实现步骤：

1. 实现红黑树
2. 封装set和map框架，解决KeyOFT
3. iterator
4. const_iterator
5. 解决key不支持修修改的问题
6. operator[]

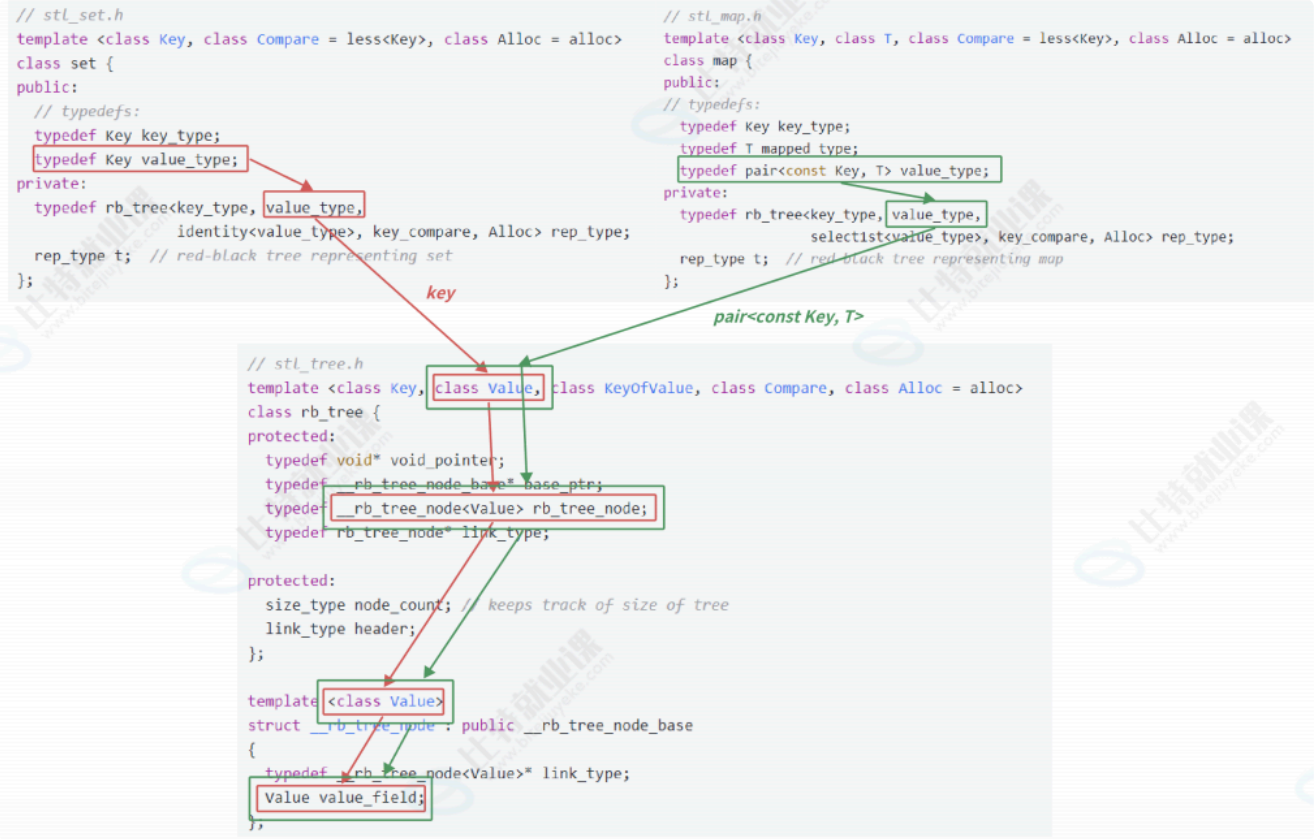
1. 源码分析

1. 我们说分析源码的第一步一定是先提炼出核心，通过我们对源码的一个分析。我们可以看出源码中的红黑树结构用了一个巧妙的泛型思想实现，源码中的红黑树是实现`key`的搜索场景，还是`key/value`的搜索场景这不是写死的，而是由第二个模板参数`Value`来决定底层红黑树节点中存储的到底是`Key`还是`Key/Value`
2. `set`实例化红黑树时第二个模板参数给的时`key`，`map`实例化红黑树时第二个模板参数给的是`pair<const key,T>`,这样一棵红黑树既可以实现`key`搜索场景`set`,也可以实现

key/value搜索场景的map。

3. 红黑树的第二个模板参数Value已经控制了红黑树节点中的存储类型，为什么还要传第一个模板参数Key呢？特别对于set,两个模板参数是一致的。这里就需要我们注意的是对于map和set，find和erase时函数参数都是Key,所以第一个模板参数是传递给find/erase等函数作形参类型的。虽然对于set而言两个参数是一样的，但是对于map而言insert的是pair对象，find和ease的是Key对象。

> 这里需要注意的是:源码模板里面模板参数是用T代表value,而内部写的value_type不是我们日常key/value场景说的value,源码中的value_type反而是红黑树结点中存储真实数据的类型。



2. 第一层：封装红黑树

2.1 复用红黑树框架

因为红黑树实现了泛型不知道T参数导致是K，还是pair<K,V>,这样的话我们如果想要在insert内部实现插入逻辑进行比较时就没有办法进行比较，因为pair的默认支持是key和value一起参与比较，我们需要任何时候都只比较key，这就需要我们在实现map和set的时候分别实现一个MapKeyOfT和SetKeyOfT,然后RBTree中通过KeyOfT仿函数取出T类型对象中的key再进行比较。

```
enum Color
{
    RED,
    BLACK
};
template <class K, class T, class KeyOfT>
class RBTree
{
    typedef RBTreeNode<T> Node;

public:
    typedef TreeIterator<T, T&, T*> Iterator;
    typedef TreeIterator<T, const T&, const T*> ConstIterator;
    //......
    private:
    Node* _root = nullptr;
    void RotateR(Node* parent)
    {
        Node* subL = parent->_left;
        Node* subLR = subL->_right;

        parent->_left = subLR;
        if (subLR)
            subLR->_parent = parent;

        subL->_right = parent;
        Node* parentparent = parent->_parent;
        parent->_parent = subL;

        if (parentparent == nullptr)
        {
            _root = subL;
            _root->_parent = nullptr;
        }
        else
        {
            if (parentparent->_left == parent)
                parentparent->_left = subL;
            else
                parentparent->_right = subL;
            subL->_parent = parentparent;
        }
    }

    void RotateL(Node* parent)
    {
```

```
        Node* subR = parent->_right;
        Node* subRL = subR->_left;

        parent->_right = subRL;
        if (subRL)
            subRL->_parent = parent;

        subR->_left = parent;
        Node* parentparent = parent->_parent;
        parent->_parent = subR;

        if (parentparent == nullptr)
        {
            _root = subR;
            _root->_parent = nullptr;
        }
        else
        {
            if (parentparent->_left == parent)
                parentparent->_left = subR;
            else
                parentparent->_right = subR;
            subR->_parent = parentparent;
        }
    }
};
```

2.2 改造insert支持插入

直接调修改红黑树的`insert`,将其修改为泛型。
解决问题：`pair` 如何比较大小？`map` 和 `set` 比较大小的方式是不同的。对于 `pair` 而言，库里面默认的比较方式是不符合要求的，因为我们要求只按照第一个值来比较，而库里面的比较是任意一个成员小就算小。这个时候我们需要自己实现一个 `pair` 去重载吗？但实际上库里面用的也是库里面的 `pair`，不是我们自己实现的。库里面调用的
是 `insert_unique` 和 `insert_equal`，`KeyOfValue` 会通过 `identify` 来取出 `key`。也就是说，传 `pair` 时需要配套一个仿函数，用来把 `pair` 的 `first` 返回作为比较依据，而不是直接用 `pair` 本身的比较运算符。

```
#pragma once
#include<iostream>
using namespace std;
enum Color
{
```

```
    RED,
    BLACK
};
template < class T>
struct RBTreeNode
{
    // 这里更新控制平衡也要加入parent指针
    T _data;
    RBTreeNode<T>* _left;
    RBTreeNode<T>* _right;
    RBTreeNode<T>* _parent;
    Color _col;
    RBTreeNode(const T& data)
        :_data(data)
        , _left(nullptr)
        , _right(nullptr)
        , _parent(nullptr)
        , _col(RED)
    {
    }
};
template < class K, class T,class KeyOfT>
class RBTree
{
    typedef RBTreeNode<T> Node;
public:
    bool Insert(const T& data)
    {
        if (_root == nullptr)
        {
            _root = new Node(data);
            _root->_col = BLACK;
            return true;
        }
        KeyOfT kot;
        Node* cur = _root;
        Node* parent = nullptr;

        // 1. 找到插入位置
        while (cur)
        {
            if (kot(cur->_data) > kot(data))
            {
                parent = cur;
                cur = cur->_right;
            }
```

```
            else if (kot(cur->_data)->kot(data))
            {
                parent = cur;
                cur = cur->_left;
            }
            else
            {
                return false;  // 键已存在
            }
        }

        // 2. 插入新节点
        cur = new Node(data);
        Node* newNode = cur;
        cur->_col = RED;
        if (kot(parent->_data) < kot(data))
        {
            parent->_right = cur;
        }
        else
        {
            parent->_left = cur;
        }
        cur->_parent = parent;
        //......
        return true;
    }
}
```

2.3 在底层实现迭代器迭代器

2.3.1 定义迭代器结构

红黑树迭代器封装的关键要点（源码分析）

1. `begin()` **返回的是中序遍历的第一个节点（最左节点），不是最大节点。**
2. `++` **操作要找到中序遍历的下一个节点（左 → 根 → 右）：**
   - 如果当前节点有右子树，则下一个节点是右子树的最左节点。
   - 如果当前节点没有右子树，则向上查找：

> 1.如果当前节点是父亲的左孩子，下一个节点就是父亲。
> 2.如果当前节点是父亲的右孩子，则继续向上，直到找到一个是其父亲左孩子的祖先，该祖先就是下一个节点。

> 3.如果一直回到根都没找到，说明已经遍历完，此时 `_node = nullptr`，即 `end()`。

3. `end()` **用** `nullptr` **表示迭代器终点。**
   库里面更常见的做法是用一个**哨兵位节点**（比如头结点）来充当 `end`，但用 `nullptr` 是简化且可行的。
4. **对于没有实例化的类模板中的内嵌类型，必须加上** `typename`，告诉编译器这是一个类型，否则编译无法通过。
5. `begin()` **用最左节点构造迭代器，** `end()` **用** `nullptr` **构造迭代器**（或哨兵位）。
6. `--` **操作与** `++` **对称**，规则如下：
   - 如果当前节点有左子树，则上一个节点是左子树的最右节点。
   - 如果当前节点没有左子树，则向上查找：

   > 1.如果当前节点是父亲的右孩子，上一个节点就是父亲。
   > 2.如果当前节点是父亲的左孩子，则继续向上，直到找到一个是其父亲右孩子的祖先，该祖先就是上一个节点。
   > 3.如果找不到，说明是第一个节点，返回 `nullptr`。

7. **关于** `map` **的迭代器** `key` **不可修改的问题：**
   - 在红黑树底层，将节点存储的 `key` 类型设置为 `const K`（例如 `pair<const K, V>`）。
   - 这样实例化红黑树时，`key` 就是只读的，无法通过迭代器修改。

   > 注意：模板参数只要不同，就会生成不同的类模板实例，因此普通迭代器和 `const` 迭代器可以分别实例化。

```
// 迭代器封装
template<class T, class Ref, class Ptr>
struct TreeIterator
{
    typedef RBTreeNode<T> Node;
    typedef TreeIterator<T, Ref, Ptr> Self;

    Node* _node;

    TreeIterator(Node* node)
        : _node(node)
    {
    }

    Ref operator*()
    {
        return _node->_data;
```

```
    }

    Ptr operator->()
    {
        return &_node->_data;
    }

    bool operator==(const Self& s) const
    {
        return _node == s._node;
    }

    bool operator!=(const Self& s) const
    {
        return _node != s._node;
    }

    Self& operator++()
    {
        if (_node == nullptr)
            return *this;  // 或 assert

        // 情况1：存在右子树
        if (_node->_right)
        {
            Node* cur = _node->_right;
            while (cur->_left)
                cur = cur->_left;
            _node = cur;
            return *this;
        }

        // 情况2：不存在右子树，向上找
        Node* cur = _node;
        Node* parent = cur->_parent;
        while (parent && cur == parent->_right)
        {
            cur = parent;
            parent = parent->_parent;
        }
        _node = parent;
        return *this;
    }

    Self operator--()
    {
```

```
        if (_node->_left)
        {
            Node* cur = _node->_left;
            while (cur->_right)
                cur = cur->_right;
            return Self(cur);
        }
        else
        {
            Node* cur = _node;
            Node* parent = cur->_parent;
            if (parent == nullptr)
                return Self(nullptr);

            if (cur == parent->_right)
                return Self(parent);

            while (parent && cur != parent->_right)
            {
                cur = parent;
                parent = parent->_parent;
            }
            return Self(parent);
        }
    }
};
```

2.3.2 在红黑树中定义begin以及end:

```
typedef TreeIterator<T, T&, T*> Iterator;
typedef TreeIterator<T, const T&, const T*> ConstIterator;

Iterator Begin()
{
    if (_root == nullptr)
        return Iterator(nullptr);
    Node* min = _root;
    while (min->_left)
        min = min->_left;
    return Iterator(min);
}

Iterator End()
{
```

```
    return Iterator(nullptr);
}

ConstIterator Begin() const
{
    if (_root == nullptr)
        return ConstIterator(nullptr);
    Node* min = _root;
    while (min->_left)
        min = min->_left;
    return ConstIterator(min);
}

ConstIterator End() const
{
    return ConstIterator(nullptr);
}
```

2.3.3 mymap以及myset下红黑树的封装

以map为例子：

```
typedef typename RBTree<K, pair<const K, V>, MapKeyOfT>::Iterator iterator;
iterator begin() {
    return _t.Begin();
}
iterator end() {
    return _t.End();
}
```

3. 第二层：实现myset与mymap

3.1 mymap封装operate[]

1. 在插入，返回该位置迭代器
2. 不在就查找，返回对应位置的迭代器
3. 首先就要修改pair的迭代器，要注意保留位置，便于红黑树本身的修改
4. 注意返回的value底层套用的是两层迭代器

```
    V& operator[](const K& key) {
        //复用insert
        pair<iterator, bool> ret = Insert({ key,V() });//这里的V是创建一个零时
对象
        return ret.first->second;
}
```

3.2 mymap的具体实现

代码如下：

```
#pragma once
#include"RBTree.h"
namespace ZL {
    template<class K,class V>
    class Map {
        struct MapKeyOfT {
            const K& operator()(const pair<K, V>& kv) {//pair的第一个值作为返
回值，且key作为主键不能被修改
                return kv.first;
            }
        };
    public:
            typedef typename RBTree<K, pair<const K, V>,
MapKeyOfT>::Iterator iterator;
            iterator begin() {
                return _t.Begin();
            }
            iterator end() {
                return _t.End();
            }
            pair<iterator,bool> Insert(const pair<const K, V>& kv) {
                return _t.Insert(kv);
        }
            V& operator[](const K& key) {
                //复用insert
                pair<iterator, bool> ret = Insert({ key,V() });//这里的V是创
建一个零时对象
                return ret.first->second;
        }
        private:
            RBTree<K, pair<const K, V>, MapKeyOfT> _t;
```

```
    };
}
```

3.3 myset的具体实现

具体代码实现：

```
#pragma once
#include "RBTree.h"

namespace ZL
{
    template<class K>
    class Set
    {
        struct SetKeyOfT
        {
            const K& operator()(const K& key) const
            {
                return key;
            }
        };
    public:
        typedef typename RBTree<K, K, SetKeyOfT>::Iterator iterator;

        iterator begin()
        {
            return _t.Begin();
        }

        iterator end()
        {
            return _t.End();
        }

        pair<iterator, bool> Insert(const K& k)
        {
            return _t.Insert(k);
        }

    private:
        RBTree<K, K, SetKeyOfT> _t;
```

```
    };
}
```

4. 第四层代码测试

测试代码如下：

```
#pragma once
#include"mymap.h"
#include"myset.h"
void test_set() {
    ZL::Set<int> s;
    s.Insert(4);
    s.Insert(1);
    s.Insert(3);
    s.Insert(12);
    s.Insert(5);
    s.Insert(8);
    s.Insert(-1);
    ZL::Set<int>::iterator it = s.begin();
    while (it != s.end()) {
        cout << *it << " ";
        ++it;
    }
    cout << endl;
}
void text_map() {
    ZL::Map<string, string> dict;
    dict.Insert({ "sort", "排序" });
    dict.Insert({ "left", "排序" });
    dict.Insert({ "right", "排序" });
    dict["left"] = "左边，剩余";//修改
    dict["insert"] = "插入";//修改+修改
    dict["string"];//插入
    ZL::Map<string, string>::iterator it = dict.begin();
    while (it != dict.end()) {
        it->second += 'x';
        cout << it->first << ":" << it->second << endl;
        ++it;
    }
    cout << endl;
}
int main() {
    //test_set();
```

```
    text_map();
    return 0;
}
```